Επιστρέφει 0 αν αυτό το String είναι ίσο με το anotherString, αρνητικό ακέραιο αν αυτό το String είναι λεξικογραφικά μικρότερο από το anotherString και θετικό ακέραιο, αν αυτό το String είναι μεγαλύτερο. Για παράδειγμα, η πρόταση

```
System.out.println((s1 + " Black").compareTo(s2));
```

θα επιστρέψει θετικό ακέραιο. Πιο συγκεκριμένα, η αξιολόγηση της έκφρασης μέσα στην println γίνεται ως εξής: Πρώτα αξιολογείται η σύνδεση s1+" Black" που αποκτά προτεραιότητα καθώς περικλείεται σε παρένθεση. Το αποτέλεσμα αυτής της σύνδεσης είναι το String John Black. Το String αυτό συγκρινόμενο με την παράμετρο s2 που έχει περιεχόμενο John θεωρείται λεξικογραφικά μεγαλύτερο.

```
int compareToIgnoreCase(String anotherString)
```

Συγκρίνει αυτό το String με την παράμετρό της χωρίς να λαμβάνει υπόψη τυχόν διαφορές μεταξύ πεζών και κεφαλαίων. Για παράδειγμα, η κλήση

```
s1.compareToIgnoreCase("john");
```

επιστρέφει 0, παρότι το s1 έχει ως περιεχόμενο το John.

```
boolean endsWith(String suffix)
```

Επιστρέφει true εφόσον η παράμετρος suffix είναι το τελευταίο τμήμα αυτού του String. Αντίστοιχα, λειτουργεί και η startsWith.

```
boolean equalsIgnoreCase(String anotherString)
```

Επιστρέφει true εφόσον αυτό το String είναι ίσο με την παράμετρό της χωρίς να λαμβάνονται υπόψη διαφορές μεταξύ πεζών και κεφαλαίων. Η συνάρτηση μοιάζει πολύ με την compareToIgnoreCase με τη διαφορά ότι είναι boolean, ενώ η compareToIgnoreCase επιστρέφει int. Βεβαίως, όπως όλες οι κλάσεις της Java, η κλάση String υποστηρίζει μέθοδο equals.

```
int indexOf(String str)
```

Επιστρέφει τη θέση της str μέσα σε αυτό το String. Αν η str δεν βρεθεί, επιστρέφει -1.
Η συνάρτηση lastIndexOf(String str) επιστρέφει την τελευταία θέση της str μέσα σε αυτό το String. Αν η str δεν βρεθεί, επιστρέφει -1.

```
int indexOf(String str, int fromIndex)
```

Επιστρέφει τη θέση της str μέσα σε αυτό το String εφόσον αυτό βρεθεί από τη θέση fromIndex και μετά, διαφορετικά, επιστρέφει -1.

```
int length()
```

Επιστρέφει το μήκος αυτού του String.

```
boolean isEmpty()
```

Επιστρέφει true μόνον εφόσον το μήκος του String είναι ίσο με το 0.

```
boolean isBlank()
```

Επιστρέφει true αν το μήκος του String είναι 0 ή αν το String περιέχει μόνο λευκούς χαρακτήρες (whitespace characters). Σημειώστε πως λευκοί χαρακτήρες είναι όσοι χαρακτήρες ή σειρές χαρακτήρων αντιπροσωπεύουν τυπογραφικά οριζόντιο ή κάθετο διάστημα. Για παράδειγμα, ο κώδικας

```
System.out.println(s.length()+"                "+s.isEmpty()+"
"+s.isBlank());
```

θα τυπώσει

```
0 true true
```

εφόσον, s==”” και

```
1 false true
```

αν s==” ”;

```
String repeat(int count)
```

Επιστρέφει ένα String με τιμή την τιμή αυτού του String επαναλαμβανόμενη count φορές. Για παράδειγμα, η κλήση

```
System.out.println(s1.repeat(2));
```

θα εμφανίσει JohnJohn.

```
String replace (String s1, String s2)
```

Είναι υπερφορτωμένη ώστε να δέχεται ως παραμέτρους String ή char. Αντικαθιστά σε αυτό το String την πρώτη εμφάνιση της πρώτης παραμέτρου της με τη δεύτερη παράμετρο. Για παράδειγμα,

```
String s3 = "John came home yesterday";
System.out.println(s3=s3.replace("John", "Mary"));
System.out.println(s3.replace('M', 'm'));
```

Η πρώτη κλήση της replace λαμβάνει τα Strings John και Mary και επιστρέφει ένα String ίδιο με το s3 στο οποίο το τμήμα John έχει αντικατασταθεί απο το Mary. Η δεύτερη κλήση λαμβάνει δύο χαρακτήρες και αντικαθιστά το M με το m. Προσοχή, καμία κλήση της replace δεν μεταβάλλει το s3. Σε όλες τις περιπτώσεις επιστρέφεται ένα νέο String. Όπως εξηγείται στην ενότητα 12.1.1, αυτό ισχύει για όλες τις συναρτήσεις της κλάσης String.

```
String[] split(String regex)
```

Λαμβάνει ως παράμετρο ένα String που αναπαριστά μια κανονική έκφραση. Οι κανονικές εκφράσεις είναι έξω από την ύλη αυτού του βιβλίου. Μας είναι χρήσιμο όμως να γνωρίζουμε ότι ο χαρακτήρας διάστημα μπορεί να χρησιμοποιηθεί ως όρισμα στη συνάρτηση. Στην περίπτωση αυτή, η συνάρτηση επιστρέφει έναν πίνακα από Strings που συνιστούν τμήματα αυτού του String διαχωρισμένα με τον χαρακτήρα διάστημα. Για παράδειγμα, ο κώδικας

```
String s3 = "John came home yesterday";
String[] t = s3.split(" ");
System.out.println(Arrays.asList(t));
```

θα τυπώσει

```
[John, came, home, yesterday]
```

```
String stripLeading()
```

Διαγράφει όλους τους λευκούς χαρακτήρες από την αρχή αυτού του String. Για παράδειγμα, ο κώδικας

```
String s5 = "   John";
System.out.println(s5.stripLeading().equals("John"));
```

τυπώνει true. Αντίστοιχα λειτουργεί η stripTrailing που διαγράφει τους λευκούς χαρακτήρες από το τέλος αυτού του String. Η δε συνάρτηση trim() διαγράφει τους λευκούς χαρακτήρες από την αρχή και από το τέλος αυτού του String.

```
String substring(int beginIndex)
String substring(int beginIndex, int endIndex)
```

Επιστρέφει ένα τμήμα αυτού του String. Είναι υπερφορτωμένη. Το τμήμα που επιστρέφεται αρχίζει από την θέση beginIndex έως το τέλος του String ή έως και τη θέση endIndex-1, ανάλογα με τις πραγματικές παραμέτρους. Για παράδειγμα, ο κώδικας

```
System.out.println(s3.substring(5));
System.out.println(s3.substring(5, 10));
```

εμφανίζει

```
came home yesterday
came
```

Τέλος, οι συναρτήσεις toLowerCase() και toUpperCase() επιστρέφουν το String που τις καλεί με πεζά ή κεφαλαία, αντίστοιχα.

### 12.1.1 Αμεταβλητότητα

Τα αντικείμενα της κλάσης String είναι αμετάβλητα (immutable). Αυτό σημαίνει πως από τη στιγμή της δημιουργίας τους μέχρι το τέλος της εφαρμογής δεν είναι δυνατόν να μεταβληθεί το περιεχόμενό τους. Καμία από τις συναρτήσεις που συζητήσαμε παραπάνω δεν μεταβάλλει το περιεχόμενο ενός String. Για παράδειγμα, ο κώδικας

```
String s6 = new String("JOHN");
s6 = s6.toLowerCase();
```

έχει ως αποτέλεσμα να δημιουργηθούν τα Strings JOHN και john. Αρχικά, η μεταβλητή s6 αναφέρεται στο JOHN, στη συνέχεια όμως ανακατευθύνεται στο john και το JOHN μένει χωρίς αναφορά και επομένως είναι υποψήφιο για να συλλεγεί από τον συλλέκτη απορριμάτων.

Τα Strings είναι αμετάβλητα ώστε να είναι ασφαλή από την επίδραση διαφορετικών νημάτων (thread safe). Σε μια πολυνηματική εφαρμογή, διαφορετικά προγράμματα μοιράζονται τις ίδιες μεταβλητές. Έτσι είναι δυνατόν ένα πρόγραμμα να μεταβάλει την τιμή ενός String και να δημιουργήσει προβλήματα σε ένα άλλο πρόγραμμα που χρησιμοποιεί το ίδιο String.

Προσέξτε πως ο αμετάβλητος χαρακτήρας των String καθιστά τον σταδιακό υπολογισμό τους ιδιαίτερα κοστοβόρο. Για παράδειγμα, η συνάρτηση

```
static String buildString() {
     String rVal = "";
     for (char c = 'A'; c <= 'Z'; c++) {
          rVal += c;
     }
     return rVal;
}
```

δημιουργεί καταρχάς ένα κενό String. Στη συνέχεια μέσα στον βρόγχο for προσθετει έναν-έναν τους χαρακτήρες του αγγλικού αλφάβητου. Κάθε φορά που προστίθεται ένας χαρακτήρας δημιουργείται ένα νέο String, δηλαδή η συνάρτηση δημιουργεί ένα σύνολο από 27 συνολικά Strings ενώ υποτίθεται ότι χρειάζεται μόνο ένα.

## 12.2 Η κλάση StringBuilder

Τα αντικείμενα της κλάσης StringBuilder συνιστούν μεταβλητές συμβολοσειρές. Έτσι, όποτε χρειαζόμαστε να διαμορφώσουμε σταδιακά μια συμβολοσειρά, είναι προτιμότερο να χρησιμοποιούμε ένα αντικείμενο

StringBuilder αντί για String. Στη συνέχεια, αφού ολοκληρώσουμε τη διαμόρφωση της συμβολοσειράς μπορούμε να δημιουργήσουμε ένα αντικείμενο String.

```
static String buildString() {
    StringBuilder rVal = new StringBuilder(26);
    for (char c = 'A'; c <= 'Z'; c++) {
        rVal.append(c);
    }
    return new String(rVal);
}
```

***Κώδικας 12.2*** *Διαμόρφωση συμβολοσειράς με την StringBuilder*

Στον κώδικα 12.2 δημιουργούμε καταρχάς ένα αντικείμενο, rVal, τύπου StringBuilder με χωρητικότητα 26 χαρακτήρων. Στη συνέχεια προσθέτουμε έναν-έναν τους χαρακτήρες του αγγλικού αλφάβητου. Κάθε εκτέλεση της append τοποθετεί έναν χαρακτήρα στην πρώτη διαθέσιμη θέση του εσωτερικού αποθηκευτικού χώρου του rVal. Αφού η συμβολοσειρά φτάσει σε μια τελική μορφή, δημιουργούμε και επιστρέφουμε ένα String.

Στις περιπτώσεις που δεν γνωρίζουμε εκ των προτέρων το απαιτούμενο μήκος, μπορούμε να χρησιμοποιήσουμε τον δημιουργό χωρίς παραμέτρους της κλάσης StringBuilder. Ο δημιουργός χωρίς παραμέτρους κατασκευάζει ένα αντικείμενο αρχικής χωρητικότητας 16 χαρακτήρων. Έτσι κι αλλιώς, αν κατά την εκτέλεση του προγράμματος προκύψουν αυξημένες απαιτήσεις, ο εσωτερικός αποθηκευτικός χώρος αυξάνεται αυτόματα.

Ένας επιπλέον δημιουργός που μας ενδιαφέρει λαμβάνει ως παράμετρο ένα String με βάση το οποίο κατασκευάζει το αντικείμενο StringBuilder.

Η κλάση διαθέτει τις μεθόδους, charAt, compareTo, indexOf, lastIndexOf και substring που λειτουργούν παρόμοια με τις αντίστοιχες μεθόδους της κλάσης String. Έχει όμως και άλλες μεθόδους, κυριότερες από τις οποίες είναι:

```
int capacity()
```

Επιστρέφει το μέγεθος του εσωτερικού αποθηκευτικού χώρου αυτού του αντικειμένου

```
StringBuilder delete(int start, int end)
```

Διαγράφει τους χαρακτήρες από τη θέση start έως και τη θέση end-1 αυτού του αντικειμένου.

```
StringBuilder deleteCharAt(int index)
```

Διαγράφει τον χαρακτήρα στη θέση index αυτού του αντικειμένου.

```
StringBuilder  insert(int index, String str)
```

Εισάγει το str στη θέση index.

```
StringBuilder replace(int start, int end, String str)
```

Αντικαθιστά τους χαρακτήρες από τη θέση start έως την end-1 με το str.

```
StringBuilder reverse()
```

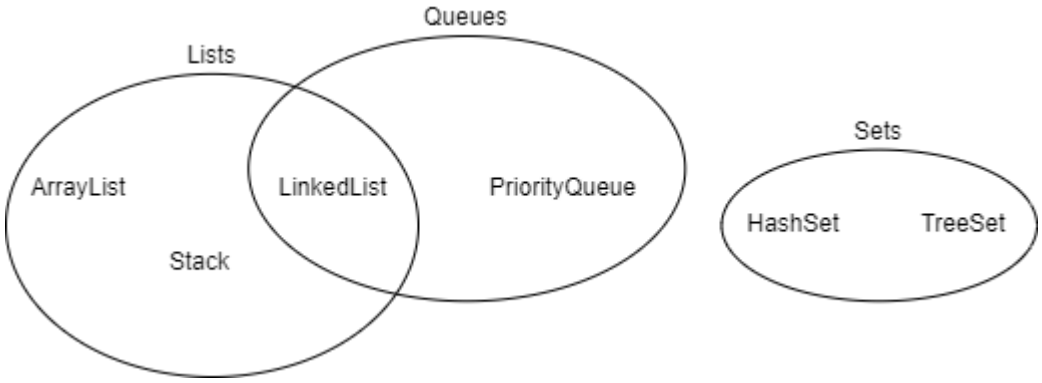
Αντιστρέφει τη σειρά των χαρακτήρων.

Προσοχή, η συνάρτηση equals της κλάσης δεν συγκρίνει δύο αντικείμενα ως προς το περιεχόμενό τους αλλά συγκρίνει τις διευθύνσεις τους, δηλαδή η equals επιστρέφει true μόνον εφόσον δύο μεταβλητές τύπου StringBuilder αναφέρονται στην ίδια θέση μνήμης.

Για περισσότερες λεπτομέρειες ανατρέξτε στην τεκμηρίωση της κλάσης.

## 12.3 Συλλογές

Σε αυτήν την ενότητα παρουσιάζουμε τις συλλογές (Collections), μια σημαντική κατηγορία κλάσεων της τυπικής βιβλιοθήκης της Java. Οι συλλογές είναι κλάσεις στα αντικείμενα των οποίων μπορούμε να αποθηκεύουμε άλλα αντικείμενα. Οι συλλογές χωρίζονται σε τρεις κύριες υποομάδες, τις Λίστες (Lists), τις Ουρές (Queues) και τα Σύνολα (Sets).




***Σχήμα 12.1*** *Οι συλλογές της Java*

Πιο συγκεκριμένα, θα εξετάσουμε τις κλάσεις ArrayList, Stack, LinkedList, PriorityQueue, HashSet και TreeSet. Όπως φαίνεται στο σχήμα 12.1, η ArrayList, η Stack και η LinkedList ανήκουν στην ομάδα των Λιστών. Η δε LinkedList ανήκει και στην ομάδα των Ουρών όπου ανήκει και η PriorityQueue. Στα σύνολα ανήκουν οι κλάσεις HashSet και TreeSet.

Ένα ιδιαίτερο χαρακτηριστικό αυτών των κλάσεων είναι ότι συνιστούν παραμετροποιημένους τύπους ή γενικεύσεις (generics). Οι γενικεύσεις επιτρέπουν σε ένα τύπο ή μέθοδο να επενεργεί σε αντικείμενα διαφορετικών τύπων ενώ παρέχουν έλεγχο των τύπων κατά τον χρόνο μεταγλώττισης [1]. Τα αντικείμενα ενός γενικευμένου τύπου εξειδικεύονται κατά την δημιουργία τους. Για παράδειγμα, μπορεί κανείς να δημιουργήσει ένα ArrayList που αποθηκεύει αντικείμενα της κλάσης String ή ένα ArrayList που αποθηκεύει αντικείμενα μιας κλάσης οριζόμενης από τον χρήστη, π.χ. Person. Οι γενικεύσεις παρουσιάζονται αναλυτικά στην ενότητα 17. Εδώ όμως μπορούμε να δούμε πώς χρησιμοποιούνται τουλάχιστον σε ό,τι χρειάζεται για τη μελέτη των συλλογών.

Η κλάση Collections παρέχει μια σειρά από χρήσιμες στατικές μεθόδους διαχείρισης των συλλογών. Είναι για τις συλλογές κάτι αντίστοιχο με ότι είναι για τους πίνακες η κλάση Arrays. Δεν παρουσιάζεται εδώ αλλά αντίθετα όπου γίνεται χρήση της δίνονται οι απαραίτητες επεξηγήσεις. Φυσικά, ο ενδιαφερόμενος αναγνώστης μπορεί να ανατρέξει στην τεκμηρίωση της κλάσης για περισσότερες λεπτομέρειες.

### 12.3.1 Η κλάση ArrayList

Ένα αντικείμενο ArrayList συνιστά έναν εξελιγμένο πίνακα που μπορεί να αυξομειώσει το μέγεθός του όποτε προκύπτει ανάγκη, ενώ ταυτόχρονα προσφέρει διάφορες λειτουργίες που συχνά χρειαζόμαστε σε πίνακες. Όπως αναφέραμε προηγουμένως τα αντικείμενα ArrayList δημιουργούνται για την αποθήκευση συγκεκριμένου τύπου, π.χ.

```
ArrayList<String> l1 = new ArrayList<>();
ArrayList<Person> l2= new ArrayList<>();
```

Σε αυτόν τον κώδικα, δημιουργούνται τα ArrayList, l1 και l2. Το μεν l1 εξειδικεύεται ώστε να αποθηκεύει αντικείμενα τύπου String, το δε l2 εξειδικεύεται ώστε να αποθηκεύει αντικείμενα τύπου Person. Προσέξτε πως για τη δημιουργία αντικειμένων που αποτελούν εξειδικεύσεις γενικεύσεων απαραίτητη είναι η χρήση του τελεστή διαμαντιού (diamond operator) που συμβολίζεται με <>. Αν δεν χρησιμοποιηθεί ο τελεστής διαμαντιού, τότε δημιουργείται ένας πρωταρχικός τύπος (raw type). Για παράδειγμα, στην περίπτωση του ArrayList θα δημιουργηθεί ένα αντικείμενο σε κάθε θεση του οποίου είναι δυνατόν να αποθηκευτεί οποιοσδήποτε τύπος.

Η κλάση διατηρεί εσωτερικά έναν πίνακα όπου αποθηκεύει τα αντικείμενα. Παρότι ο πίνακας αυξομειώνεται αυτόματα όποτε χρειάζεται, παρέχεται η δυνατότητα να δημιουργηθεί με συγκεκριμένο μήκος έτσι ώστε όπου είναι εφικτό να καθίσταται ο κώδικας πιο αποτελεσματικός. Για παράδειγμα, ο κώδικας που ακολουθεί δημιουργεί ένα ArrayList εσωτερικής χωρητικότητας 30.

```
ArrayList<String> l3 = new ArrayList<>(30);
```

Σε περίπτωση που δεν οριστεί η χωρητικότητα κατά τη δημιουργία του πίνακα, τότε αυτή λαμβάνει την τιμή 10.
Αν χρειάζεστε έναν πίνακα από ArrayList αντικείμενα, τότε μπορείτε να τον δημιουργήσετε ως εξής.

```
ArrayList<String>[] t = new ArrayList[10];
```

Προσέξτε πως εδώ δημιουργείται κατάλληλος πίνακας για να αποθηκευτούν 10 αντικείμενα πρωταρχικού τύπου ArrayList, καθώς δεν είναι εφικτή η χρήση του τελεστή διαμαντιού κατά τη δημιουργία πινάκων γενικευμένων τύπων. Πάντως, κανένα από αυτά τα αντικείμενα δεν έχει δημιουργηθεί ακόμη. Έχει δημιουργηθεί μόνο ο πίνακας κάθε θέση του οποίου έχει τεθεί στο null. Στη συνέχεια, όμως, μπορείτε να δημιουργήσετε ένα αντικείμενο για κάθε θέση του πίνακα t, χρησιμοποιώντας τον τελεστή διαμαντιού όπως φαίνεται αμέσως από κάτω.

```
t[0]=new ArrayList<>(30);
```

Χρησιμοποιήστε ένα ArrayList εκεί που χρειάζεστε ταχεία πρόσβαση στα στοιχεία μιας λίστας. Ωστόσο, λάβετε υπόψη πως η πρόσθεση ή η διαγραφή στοιχείων από ενδιάμεση θέση ενός ArrayList είναι σχετικά χρονοβόρες διαδικασίες. Αν ένα στοιχείο θα πρέπει να διαγραφεί, π.χ., από το μέσον της λίστας, τότε ο εσωτερικός πίνακας θα πρέπει να μετακινήσει κατάλληλα όλα τα στοιχεία από το μέσον και μετά.

```
static void usage() {
    ArrayList<String> l = new ArrayList<>();
    l.add("1");
    l.add("2");
    l.add("4");
    l.add(2, "3");

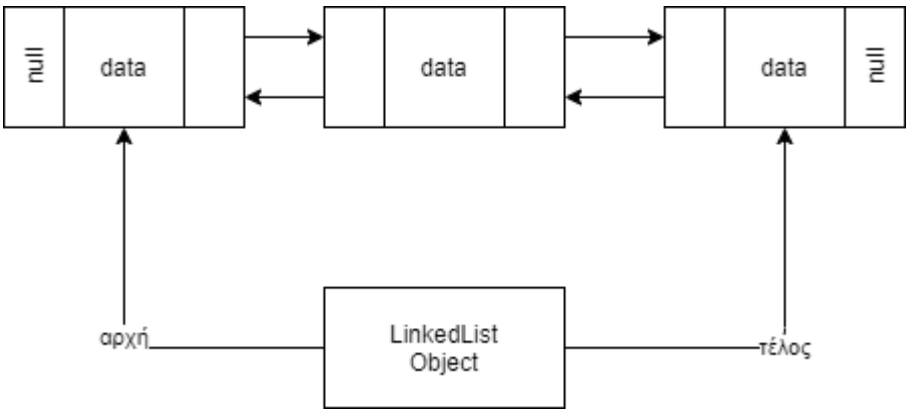
    for (int idx = 0; idx < l.size(); idx++) {
        System.out.print(l.get(idx));
    }
    System.out.print("-");
    l.remove("2");
    System.out.println(l);
}
```

***Κώδικας 12.3*** *Παράδειγμα χρήσης της ArrayList*

Στον κώδικα 12.3, παρατίθεται παράδειγμα χρήσης της ArrayList. Καταρχάς, δημιουργούμε ένα ArrayList κατάλληλο για την αποθήκευση αντικειμένων της κλάσης String. Στη συνέχεια εισάγουμε τα Strings 1, 2 και 4. Στη συνέχεια, τοποθετούμε String με τιμή 3 στη θέση 2, δηλαδή ανάμεσα στις θέσεις των σειρών 2 και 4. Αμέσως μετά προσπελαύνουμε όλα τα στοιχεία της l με δεικτοδοτημένη for. Διαγράφουμε ένα στοιχείο της και την ξανατυπώνουμε. Οι κλήσεις l.add(2, "3") και l.remove("2") είναι οι πιο χρονοβόρες λειτουργίες στον κώδικα 12.3. Ο κώδικας εμφανίζει 1234-[1, 3, 4].

### 12.3.2 Η κλάση LinkedList

Τα αντικείμενα LinkedList συνιστούν αμφίδρομες λίστες. Επομένως, κάθε στοιχείο μίας τέτοιας λίστας, πλέον του αντικειμένου που αποθηκεύει, αποθηκεύει και τις διευθύνσεις του προηγούμενου και του επόμενου στοιχείου, όπως φαίνεται στο σχήμα 12.2.




***Σχήμα 12.2*** *Δομή της κλάσης LinkedList*

Σε μία τέτοια λίστα μπορεί κανείς να προσθέσει ή να αφαιρέσει στοιχεία ταχύτατα, καθώς οι λειτουργίες αυτές δεν απαιτούν μετακίνηση των στοιχείων. Αντίθετα, αργή είναι η διαδικασία προσπέλασης ενός συγκεκριμένου στοιχείου καθώς η αναζήτησή του αρχίζει αναγκαστικά από την αρχή ή το τέλος της λίστας και είναι σειριακή.

Όπως φαίνεται στο σχήμα 12.1, οι λίστες αυτές είναι ταυτοχρόνως και ουρές, δηλαδή μπορεί να χρησιμοποιηθούν ως First In – First Out (FIFO) δομές. Αυτό επιτυγχάνεται εύκολα όταν προσθέτουμε στοιχεία στο ένα άκρο της λίστας και τα αφαιρούμε από το άλλο. Επιπλέον, είναι προφανές ότι μπορούν να χρησιμοποιηθούν και σαν δομές LIFO.

```
static void usage() {
    LinkedList<String> l = new LinkedList<>();
    l.addFirst("1");
    l.addFirst("2");
    l.addLast("3");
    l.set(2, "4");
    System.out.print(l.peekFirst());
    System.out.print("-");
    while (!l.isEmpty()) {
        System.out.print(l.pollLast());
    }
}
```

***Κώδικας 12.4*** *Παράδειγμα χρήσης LinkedList*

Στον κώδικα 12.4, καταρχάς δημιουργούμε ένα αντικείμενο LinkedList κατάλληλο για να αποθηκεύουμε Strings. Στη συνέχεια προσθέτουμε στην αρχή της λίστας το String 1 και μετά το 2. Επομένως η λίστα έχει τα δεδομένα της στη σειρά 2-1. Στη συνέχεια, προσθέτουμε το 3 στο τέλος της λίστας, οπότε η λίστα γίνεται 2-1-3. Μετά στη θέση 2 της λίστας, δηλαδή εκεί που είναι αποθηκευμένο το 3, τοποθετούμε το 4 και η λίστα γίνεται 2-1-4. Στη συνέχεια, με την peekFirst ανακτούμε το στοιχείο που βρίσκεται στην αρχή της λίστας, δηλαδή το 2. Η peekFirst επιστρέφει το στοιχείο στην αρχή της λίστας χωρίς να το αφαιρεί από αυτήν.

Τέλος στην επαναληπτική διαδικασία, ενόσω η λίστα δεν είναι αδεια, αφαιρούμε και εμφανίζουμε το στοιχείο στην τελευταία θέση της λίστας. Η έξοδος της useExample είναι 2-412.

### 12.3.3 Η κλάση Stack

Η κλάση Stack παρέχει αντικείμενα που προσομοιώνουν τη δομή Last In – First Out (LIFO). Οτιδήποτε εισάγεται σε ένα Stack, εισάγεται στην κορυφή του και οτιδήποτε εξάγεται, λαμβάνεται από την κορυφή του. Επομένως, ένα τέτοιο αντικείμενο προσφέρει ταχύτατη πρόσβαση στην περίπτωση που χρησιμοποιείται όντως ως δομή LIFO. Αν όμως χρειαστεί να προσπελάσουμε αντικείμενα που δεν βρίσκονται στην κορυφή του Stack, τότε πρέπει να τα αναζητήσουμε σειριακά. Βεβαίως, είναι αυτονόητο ότι προσθέσεις ή αφαιρέσεις αντικειμένων σε άλλη θέση πέραν της κορυφής δεν έχουν νόημα σε ένα Stack.

```
static void usage() {
    Stack<String> s = new Stack<>();
```

```
        s.push("1");
        s.push("2");
        s.push("3");
        s.push("4");
        while (!s.isEmpty()) {
            System.out.println(s.pop());
        }
    }
```

***Κώδικας 12.5*** *Παράδειγμα χρήσης της Stack*

Στον κώδικα 12.5, ωθούμε με την push στη στοίβα s, τέσσερα String. Στη συνέχεια αφαιρούμε από τη στοίβα, ένα-ένα τα στοιχεία της έως ότου αδειάσει. Η έξοδος της συνάρτησης θα εμφανίσει 4321.

### 12.3.4 Η κλάση PriorityQueue

Η κλάση PriorityQueue προσομειώνει μια ουρά προτεραιότητας. Τα αντικείμενα που εισάγονται στην ουρά ταξινομούνται ως προς τη φυσική σειρά τους. Αν τα αντικείμενα δεν έχουν φυσική σειρά, τότε εφαρμόζονται άλλες τεχνικές που παρουσιάζονται στην ενότητα 14.1.2. Όταν ένα αντικείμενο εξάγεται από την ουρά, εξάγεται αυτό που έχει την μεγαλύτερη προτεραιότητα, δηλαδή τη μικρότερη σειρά στην ταξινομημένη ουρά. Στην περίπτωση που όλα τα αντικείμενα στην ουρά έχουν την ίδια προτεραιότητα, επιστρέφεται ένα αντικείμενο που επιλέγεται τυχαία.

```
    static void usage() {
        PriorityQueue<String> s = new PriorityQueue<>();
        s.add("John");
        s.add("Mark");
        s.add("Ann");
        s.add("Bill");
        while (!s.isEmpty()) {
            System.out.println(s.poll());
        }
    }
```

***Κώδικας*** *12.6 Παράδειγμα χρήσης ουράς προτεραιότητας*

Στον κώδικα 12.6 εισάγουμε τέσσερα Strings στην ουρά προτεραιότητας s με τη συνάρτηση add. Στη συνέχεια τα ανακτούμε ένα προς ένα με την poll. Η έξοδος της useExample εμφανίζει τα Strings ταξινομημένα κατ' αύξουσα αλφαριθμητική σειρά. Σημειώστε πως τα αντικείμενα String έχουν φυσική σειρά.

### 12.3.5 Οι κλάσεις HashSet και TreeSet

Τρία είναι τα κύρια στοιχεία που χαρακτηρίζουν τα αντικείμενα της κλάσης HashSet. Κατά πρώτον είναι σύνολα (sets). Σε ένα σύνολο ένα στοιχείο μπορεί να υπάρχει μόνο μία φορά. Επομένως, αν επιχειρήσουμε να εισάγουμε σε ένα HashSet αντικείμενο που ήδη υπάρχει, το σύνολο παραμένει αμετάβλητο. Επιπλέον, η σειρά των στοιχείων ενός συνόλου είναι αυθαίρετη και δεν παραμένει σταθερή κατά τη διάρκεια του χρόνου. Τέλος, τα σύνολα αυτά χρησιμοποιούν τους κωδικούς κατακερματισμού των αντικειμένων που αποθηκεύουν ώστε να επιταχύνουν τις λειτουργίες της προσπέλασης, πρόσθεσης και διαγραφής.

```
    static void usage() { //Κώδικας 12.7
        HashSet<String> h = new HashSet<>();
        h.add("1");
        h.add("2");
        h.add("3");
        h.add("4");
        h.add("1");
        System.out.println(h);
        h.remove("2");
        if (h.contains("3")) {
            System.out.println("element found");
```

```
        }
        for (String s : h) {
            System.out.print(s);
        }
    }
```

***Κώδικας 12.7*** *Παράδειγμα χρήσης HashSet*

Στον κώδικα 12.7 παρατίθεται παράδειγμα χρήσης ενός HashSet. Όπως ήδη αναφέρθηκε, η πρόσθεση (add) στοιχείου και η διαγραφή (remove) είναι ταχείς διεργασίες καθώς χρησιμοποιείται ο κωδικός κατακερματισμού. Για τον ίδιο λόγο ταχύς είναι και ο έλεγχος αν το σύνολο περιέχει (contains) κάποιο στοιχείο. Τα στοιχεία αυτής της συλλογής, μπορούμε να τα προσπελάσουμε με την ενισχυμένη for, όπως δείχνει ο κώδικας 12.7. Σημειώστε πως δεν παρέχεται η δυνατότητα να ανακτήσουμε ένα στοιχείο από συγκεκριμένη θέση (get(i)), καθώς τα στοιχεία του συνόλου δεν διατηρούν σταθερή σειρά.

Παρόμοια με την κλάση HashSet είναι και η TreeSet, μόνο που δεν βασίζεται σε κωδικούς κατακερματισμού αλλά σε δυαδικά δένδρα αναζήτησης.

## 12.4 Οι κλάσεις HashMap και TreeMap

Οι κλάσεις HashMap και TreeMap ανήκουν σε ίδια ομάδα κλάσεων, στις λεγόμενες απεικονίσεις (Maps). Κάθε στοιχείο μιας απεικόνισης συσχετίζει δύο αντικείμενα, το κλειδί με την τιμή. Για παράδειγμα, έστω ότι θέλουμε να μετρήσουμε το πλήθος κάθε λέξης ενός κειμένου. Προφανώς, μας διευκολύνει μια δομή όπου η λέξη θα είναι κλειδί και το πλήθος η τιμή. Εύκολα μπορούμε να το πετύχουμε με ένα HashMap.

```
    static void usage(String text) {
        HashMap<String, Integer> hM = new HashMap<>();
        String[] words = text.split(" ");
        System.out.println(Arrays.asList(words));
        for (String word : words) {
            Integer cnt = hM.get(word);
            if (cnt==null) {
                cnt=0;
            }
            hM.put(word, cnt+1);
        }
        System.out.println(hM);
    }
```

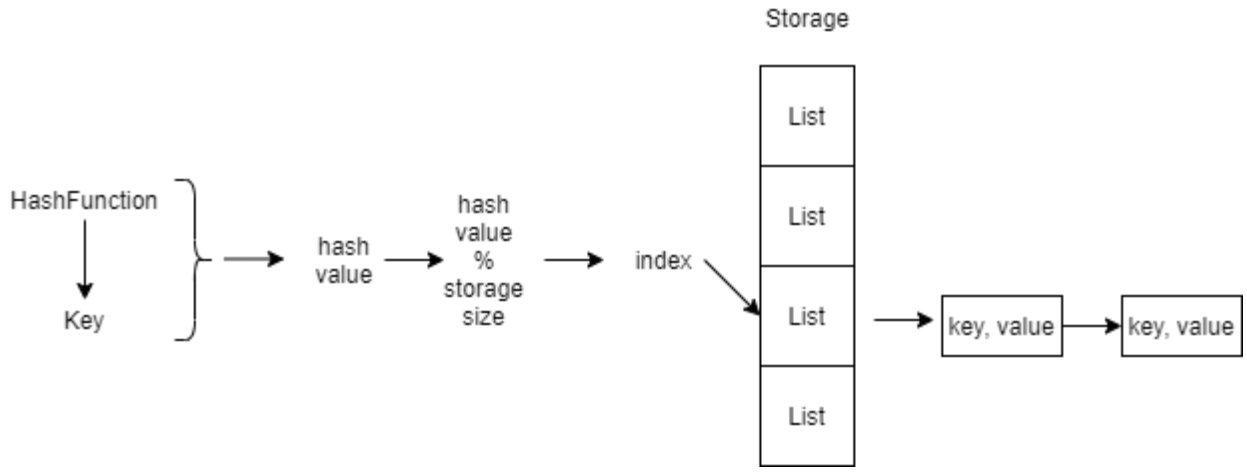
***Κώδικας 12.8*** *Μέτρηση των λέξεων κειμένου με την HashMap*

Στον κώδικα 12.8 παρουσιάζουμε ένα παράδειγμα όπου μετράμε το πλήθος κάθε λέξης του text με τη βοήθεια του HashMap, hM. Η κλάση HashMap είναι παραμετροποιημένη ως προς δύο τύπους. Ο πρώτος αναπαριστά το κλειδί (key) και ο δεύτερος την τιμή (value). Το αντικείμενο που έχουμε δημιουργήσει εδώ έχει ως κλειδί ένα String και ως τιμή έναν Integer. Κάθε εγγραφή στο hM περιλαμβάνει μια λέξη και το πλήθος που αυτή βρίσκεται στο text, δηλαδή είναι της μορφής <λέξη, πλήθος>. Προσέξτε όμως πως το πλήθος δεν θα μπορούσε να είναι τύπου int, καθώς οι τύποι των γενικεύσεων δεν μπορεί να είναι θεμελιώδεις τύποι. Μετά τη δημιουργία του hM, παίρνουμε τις λέξεις του κειμένου όπως αυτές διαχωρίζονται με διαστήματα στον πίνακα words. Για απλοποίηση, υποθέτουμε πως το κείμενο δεν περιλαμβάνει σημεία στίξης.

Στη συνέχεια, για κάθε λέξη word στον πίνακα words, ελέγχουμε με την get αν υπάρχει εγγραφή στο hM. Αν η get βρει εγγραφή μας επιστρέφει το πλήθος, αν όχι επιστρέφει null. Εφόσον δεν υπάρχει ήδη εγγραφή, αρχικοποιούμε τον μετρητή cnt στο 0. Στη συνέχεια, με την put ενημερώνουμε την εγγραφή. Προσέξτε πως αν εγγραφή με αυτό το κλειδί δεν υπάρχει ήδη, η put την εισάγει, διαφορετικά την ενημερώνει.

Χρήσιμο είναι να γνωρίζουμε τη βασική ιδέα στην οποία βασίζεται η κλάση HashMap.

Ο σκοπός της κλάσης HashMap είναι να παρέχει ταχεία πρόσβαση στα δεδομένα με βάση κάποιο κλειδί. Προς επίτευξη του σκοπού, αρχικά υπολογίζεται η τιμή hash του κλειδιού, όπως φαίνεται στο σχήμα 12.3. Ο υπολογισμός γίνεται από τη συνάρτηση hashCode του κλειδιού. Κάθε αντικείμενο HashMap διατηρεί έναν εσωτερικό πίνακα storage. Τα στοιχεία αυτού του πίνακα είναι συνδεδεμένες λίστες.




***Σχήμα 12.3*** *Η δομή HashMap*

Στη συνέχεια υπολογίζεται το ακέραιο υπόλοιπο της τιμής hash δια του μεγέθους του storage. Το αποτέλεσμα βρίσκεται υποχρεωτικά στο εύρος 0..storage.length, οπότε χρησιμοποιείται σαν δείκτης πρόσβασης στον storage. Από τη διαδικασία που περιγράφηκε μέχρι εδώ είναι προφανές πως διαφορετικά κλειδιά ενδέχεται να αντιστοιχιστούν στην ίδια θέση του storage. Σε κάθε θέση του storage όμως υπάρχει μια συνδεδεμένη λίστα. Κάθε στοιχείο της λίστας περιέχει κλειδί και τιμή. Η λίστα ανιχνεύεται προκειμένου να βρεθεί το συγκεκριμένο κλειδί, οπότε επιστρέφεται η τιμή. Η λειτουργικότητα της κλάσης TreeMap είναι παρόμοια με της HashMap. Ωστόσο, αξίζει να επισημάνουμε τις ακόλουθες διαφορές:

1. Η TreeMap βασίζεται σε δυαδικά δέντρα αναζήτησης και όχι σε συνάρτηση κατακερματισμού.
2. Η HashMap επιτρέπει ένα κλειδί με null τιμή ενώ η TreeMap όχι.
3. Η TreeMap διατηρεί τη φυσική σειρά των αντικειμένων που αποθηκεύει ενώ η HashMap όχι. Επομένως, αν η εφαρμογή μας απαιτεί πλοήγηση στα δεδομένα μιας απεικόνισης σύμφωνα με τη φυσική σειρά τους, η TreeMap είναι προτιμητέα.
4. Συνήθως η HashMap είναι ταχύτερη.
5. Η TreeMap έχει πιο πλούσια λειτουργικότητα από την HashMap, δηλαδή παρέχει περισσότερες συναρτήσεις για πλοήγηση και ανάκτηση των δεδομένων της.

## 12.5 Η κλάση BigDecimal

Όπως έχουμε επισημάνει στην ενότητα 3.2 αλλά και όπως επιδεικνύεται στον κώδικα 4.8, κατά τους υπολογισμούς με τους θεμελιώδεις τύπους float και double προκύπτουν αποκλίσεις από τα αναμενόμενα αποτελέσματα που οφείλονται στον προσεγγιστικό χαρακτήρα αυτών των τύπων.

Επομένως, οι τύποι αυτοί δεν είναι κατάλληλοι για υπολογισμούς που απαιτούν μεγάλη ακρίβεια. Για τέτοιου είδους υπολογισμούς προσφέρεται η κλάση BigDecimal. Πέραν της μεγάλης ακρίβειας, η κλάση προσφέρει και μεθόδους για στρογγυλοποίηση στα απαιτούμενα ψηφία. Ας σημειωθεί ότι τα αντικείμενα της κλάσης είναι αμετάβλητα (immutable). Η κλάση ανήκει στο πακέτο java.math.

```
static void usage() {
    BigDecimal d1 = new BigDecimal(1),
            d2 = new BigDecimal(1),
            d3 = new BigDecimal(0.00001);
    for (int j = 1; j <= 10; j++) {
        d1 = d1.add(d3);
    }
    d2 = d2.add(d3.multiply(new BigDecimal(10)));
    System.out.println(d1);
    System.out.println(d2);
    System.out.println(d1.equals(d2));
    MathContext mC = new MathContext(5);
```

```
        d1 = d1.round(mC);
        d2 = d2.round(mC);
        System.out.println(d1);
        System.out.println(d2);
        System.out.println(d1.equals(d2));
        d1=new BigDecimal(1.00016);
        System.out.println(d1.round(mC));
    }
```

***Κώδικας 12.9*** *Παράδειγμα χρήσης της BigDecimal*

Στον κώδικα 12.9 καταρχάς ορίζουμε τους BigDecimal d1, d2 και d3 με τιμές 1, 1 και 0.00001, αντίστοιχα. Στη συνέχεια, προσθέτουμε 10 φορές το d3 στο d1 και στο d2 προσθέτουμε το δεκαπλάσιο του d1. Αν τα d1, d2 και d3 ήταν float ή double, ο έλεγχος d1==d2 θα επέστρεφε false. Αντίθετα, με την BigDecimal, η σύγκριση d1.equals(d2) επιστρέφει true που είναι και η μαθηματικά ορθή τιμή. Στη συνέχεια, στρογγυλοποιούμε τα d1 και d2 χρησιμοποιώντας το αντικείμενο mC της κλάσης MathContext. Κατά την κατασκευή του mC, περνάμε ως παράμετρο τη σταθερά 5. Στη συνέχεια, η round δημιουργεί αντικείμενα με 5 ψηφία συνολικά (unscaled value). Στο παράδειγμά μας, η στρογγυλοποίηση γίνεται προς τον πλησιέστερο γείτονα (nearest neighbor). Επειδή τα 5 ψηφία ακολουθεί το 0, η στρογγυλοποίηση παράγει 1.0001. Αν όμως τα 5 ψηφία ακολουθούνται από ψηφίο μεγαλύτερο ή ίσο του 6, τότε η στρογγυλοποίηση παράγει 1.0002. Ωστόσο υπάρχει η δυνατότητα να χρησιμοποιηθεί διαφορετικό πρότυπο στρογγυλοποίησης (rounding mode). Η έξοδος του κώδικα 12.9 έχει ως εξής:

```
1.000100000000000000008180305391403130954586231382563710212707519531250
1.000100000000000000008180305391403130954586231382563710212707519531250
true
1.0001
1.0001
true
1.0002
```

## 12.6 Διαχείριση Ημερομηνίας

Στην ενότητα αυτή θα συζητήσουμε τις κλάσεις που μας διευκολύνουν στη διαχείριση ημερομηνιών. Πρόκειται για ένα σύνολο κλάσεων που ανήκουν στο πακέτο java.time. Πιο συγκεκριμένα, θα συζητήσουμε τις κλάσεις LocalDate, LocalTime, LocalDateTime, Period και Duration, ενώ θα δούμε και τον απαριθμήσιμο τύπο Month.

Ας σημειώσουμε ότι η αρχική βιβλιοθήκη διαχείρισης ημερομηνιών της Java είναι ενσωματωμένη στο πακέτο java.util. Ωστόσο η νέα βιβλιοθήκη, διαθέσιμη από την έκδοση 8 της Java, είναι πληρέστερη ώστε ο μόνος λόγος για να ασχοληθεί κανείς με την παλαιότερη είναι αν χρειάζεται να εργαστεί επάνω σε παλαιό κώδικα που την χρησιμοποιεί. Σε κάθε περίπτωση, εδώ παρουσιάζεται η νέα βιβλιοθήκη java.time.

Μια ιδιομορφία των κλάσεων που θα παρουσιάσουμε είναι ότι δεν παρέχουν δημόσιο δημιουργό. Αντ' αυτού παρέχουν κατάλληλες συναρτήσεις δημιουργίας αντικειμένων, τις αποκαλούμενες μεθόδους κατασκευής (factory methods) [2]. Αυτή η σχεδιαστική επιλογή υπαγορεύτηκε κυρίως από την υψηλή πολυπλοκότητα που απαιτεί η κλήση των δημιουργών αυτών των κλάσεων. Πράγματι, η δημιουργία ενός αντικειμένου LocalDate με χρήση δημιουργού απαιτεί σύνθετες ρυθμίσεις που έχουν να κάνουν με διαφορές μεταξύ γεωγραφικών τοποθεσιών (timezones), διαφορετικές γλώσσες, διαφορετικά ημερολόγια, κ.ά. Η χρήση κατασκευαστικών μεθόδων παρέχει μεγαλύτερη εκφραστική ισχύ με αποτέλεσμα να εξειδικεύονται και να απλοποιείται η κλήση τους. Για παράδειγμα, η κατασκευή ενός αντικειμένου που αναπαριστά τη σημερινή ημερομηνία γίνεται με απλή κλήση της συνάρτησης now() της κλάσης LocalDate.

Ένα άλλο κρίσιμο στοιχείο είναι πως τα αντικείμενα των κλάσεων αυτών είναι αμετάβλητα (immutables).

Πριν ξεκινήσουμε την περιγραφή των κλάσεων να αναφέρουμε δύο απαριθμήσιμους τύπους που παρέχει η βιβλιοθήκη. Ο τύπος DayOfWeek παρέχει σταθερές για κάθε ημέρα της εβδομάδος, από MONDAY έως SUNDAY και ο τύπος Month παρέχει σταθερές για κάθε μήνα του έτους, από JANUARY έως DECEMBER.

### 12.6.1 Η κλάση LocalDate

Ένα αντικείμενο LocalDate αναπαριστά μια ημερομηνία που συμπεριλαμβάνει ημέρα, μήνα και έτος, χωρίς να συμπεριλαμβάνει την ώρα.

```
static void usage() {//Κώδικας 12.10
    LocalDate today = LocalDate.now();
    LocalDate dec15_71 = LocalDate.of(1971, Month.DECEMBER, 15);
    LocalDate jan1_71 = LocalDate.of(1971, 1, 15);
    LocalDate tommorow = today.plusDays(1);
    LocalDate afterAWeek = today.plusWeeks(1);
    LocalDate afterAMonth = today.plusMonths(1);
    LocalDate afterAYear = today.plusYears(1);
    LocalDate yesterday = today.minusDays(1);
    LocalDate beforeAWeek = today.minusWeeks(1);
    LocalDate beforeAMonth = today.minusMonths(1);
    LocalDate beforeAYear = today.minusYears(1);
    LocalDate afterAMonthAndAWeek = today.plusMonths(1).plusWeeks(1);
}
```

***Κώδικας 12.10*** *Παράδειγμα χρήσης της LocalDate*

Όπως φαίνεται στον κώδικα 12.10, η δημιουργία ενός αντικειμένου LocalDate που αναπαριστά τη σημερινή ημερομηνία γίνεται με κλήση της στατικής μεθόδου now. Για να δημιουργήσουμε μια συγκεκριμένη ημερομηνία χρησιμοποιούμε τη στατική μέθοδο of. Στον κώδικα 12.10 δημιουργούνται δύο τέτοιες ημερομηνίες, η dec15_71 και η jan1_71. Στη θέση της δεύτερης παραμέτρου της of που αναπαριστά τον μήνα, μπορεί να χρησιμοποιηθεί int ή ο απαριθμήσιμος τύπος Month. Στη συνέχεια επιδεικνύονται οι μέθοδοι plus και minus με τις οποίες μπορούμε να προσθέτουμε ή να αφαιρούμε από μια ημερομηνία. Προσέξτε πως αυτές οι μέθοδοι δεν επηρεάζουν το αντικείμενο που τις καλεί αλλά επιστρέφουν ένα νέο αντικείμενο με το ανάλογο περιεχόμενο. Η κλάση διαθέτει αναγνώστες όπως οι getYear(), getMonth(), getDayOfWeek(), getDayOfMonth(), κλπ.

### 12.6.2 Η κλάση LocalTime

Η κλάση LocalTime αναπαριστά την ώρα, σε ώρες, λεπτά, δευτερόλεπτα και δισεκατομυριοστά του δευτερολέπτου.

```
static void usage() {
    LocalTime currentTime = LocalTime.now();
    LocalTime t12_50 = LocalTime.of(12, 50);
    LocalTime t12_50_43 = LocalTime.of(12, 50, 43);
    LocalTime t12_50_43_625 = LocalTime.of(12, 50, 43, 625);
    LocalTime after2Hours = LocalTime.now().plusHours(2);
    LocalTime after2Minutes = LocalTime.now().plusMinutes(2);
    LocalTime after2Seconds = LocalTime.now().plusSeconds(2);
    LocalTime after1MillionNanos = LocalTime.now().plusNanos(1_000_000);
    LocalTime beforeAnHour = LocalTime.now().minusHours(1);
    LocalTime before2minutes = LocalTime.now().minusMinutes(2);
    LocalTime before3Seconds = LocalTime.now().minusSeconds(3);
    LocalTime before10Nanos = LocalTime.now().minusNanos(10);
}
```

***Κώδικας 12.11*** *Παράδειγμα χρήσης της LocalTime*

Όπως φαίνεται στον κώδικα 12.11, η μέθοδος now χρησιμοποιείται για τη δημιουργία αντικειμένου που αντιπροσωπεύει την ώρα του συστήματος κατά τον χρόνο εκτέλεσης της μεθόδου. Με τη μέθοδο of μπορούμε να δημιουργήσουμε αντικείμενα που αναπαριστούν συγκεκριμένες ώρες. Κατά τη δημιουργία τέτοιων αντικειμένων πρέπει υποχρεωτικά να καθορίσουμε την ώρα και τα λεπτά και προαιρετικά, τα δευτερόλεπτα και τα νανοδευτερόλεπτα. Στη συνέχεια, γίνεται επίδειξη των μεθόδων με τις οποίες προσθέτουμε ή αφαιρούμε χρόνο από ένα αντικείμενο LocalTime. Και εδώ, όπως και στην LocalDate, το αντικείμενο που καλεί τη μέθοδο